FH University of Applied Sciences
TECHNIKUM WIEN

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

Terraform Workshop Protocol

Ausgeführt von: David Veigel
Kevin Forter
Personenkennzeichen: XXXXXXXXXXXXXXX

BegutachterIn:

Wien, den 3. Dezember 2025

Inhaltsverzeichnis

1 Workshop 1: Terraform Basics

In this workshop, we set up a simple web server on AWS using Terraform. The goal is to provision an EC2 instance in the default VPC, install an Apache web server using user data, and verify that it is reachable.

1.1 Configuration

1.1.1 Provider and Region

We configure the AWS provider to use the `us-east-1` region, as specified in the requirements.

```
provider "aws" {
  region = "us-east-1"
}
```

1.1.2 VPC and Subnets

Instead of creating a new network, we use the existing default VPC and its subnets. This simplifies the setup for this initial workshop.

```
data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default_vpc_subnets" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}
```

1.1.3 Security Group

To make our web server accessible, we create a security group. We allow inbound TCP traffic on port 80 (HTTP) from any IP address (`0.0.0.0/0`). We also allow all outbound traffic to ensure the server can download updates and packages.

```
resource "aws_security_group" "web_sg" {
  name        = "tf-web-sg"
  description = "Allow HTTP in, all egress"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description      = "HTTP"
    from_port        = 80
    to_port          = 80
    protocol         = "tcp"
    cidr_blocks      = ["0.0.0.0/0"]
  }

  egress {
    description      = "All egress"
    from_port        = 0
    to_port          = 0
    protocol         = "-1"
    cidr_blocks      = ["0.0.0.0/0"]
  }
}
```

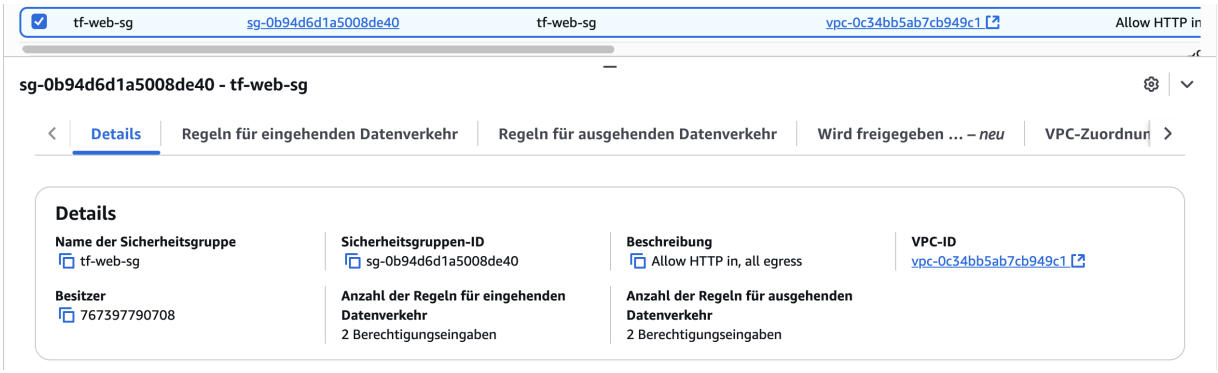


Abbildung 1: Security Group Configuration in AWS Console

1.1.4 EC2 Instance

We provision an EC2 instance using the `t2.micro` instance type, which is eligible for the free tier. We dynamically fetch the latest Ubuntu 22.04 LTS AMI. The `user_data` script is used to automatically install and start the Apache web server upon launch.

```
resource "aws_instance" "web" {
  ami                         = data.aws_ami.ubuntu.id
  instance_type               = "t2.micro"
  subnet_id                   = data.aws_subnets.default_vpc_subnets.ids[0]
```

```
  vpc_security_group_ids      = [aws_security_group.web_sg.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    sudo apt-get update
    sudo apt-get install -y apache2
    sudo systemctl start apache2
    sudo systemctl enable apache2
    echo "<h1>Hello World</h1>" | sudo tee /var/www/html/index.html
  EOF

  tags = {
    Name = "tf-apache"
  }
}
```

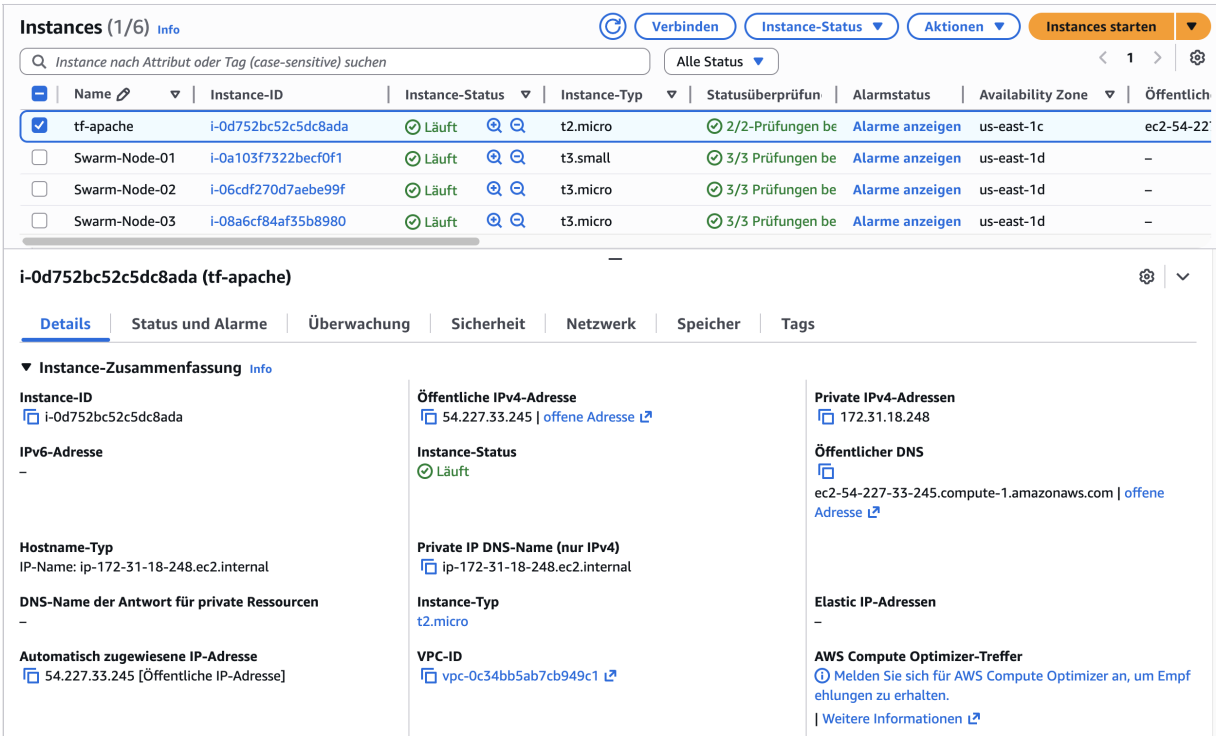


Abbildung 2: EC2 Instance Running in AWS Console

1.1.5 Output and Verification

Finally, we output the public DNS of the instance to easily access it. We also include a `null_resource` to verify that the web server is actually reachable via HTTP.

```
output "instance_public_dns" {
```

```
  description = "Public DNS of the EC2 instance"
  value       = aws_instance.web.public_dns
}
```

1.2 Execution

We initialize the Terraform project to download the necessary providers.

```
❯ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0.0"...
- Finding latest version of hashicorp/null...
- Installing hashicorp/aws v6.20.0...
- Installed hashicorp/aws v6.20.0 (signed by HashiCorp)
- Installing hashicorp/null v3.2.4...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Abbildung 3: Terraform Init

Then we apply the configuration to create the resources.

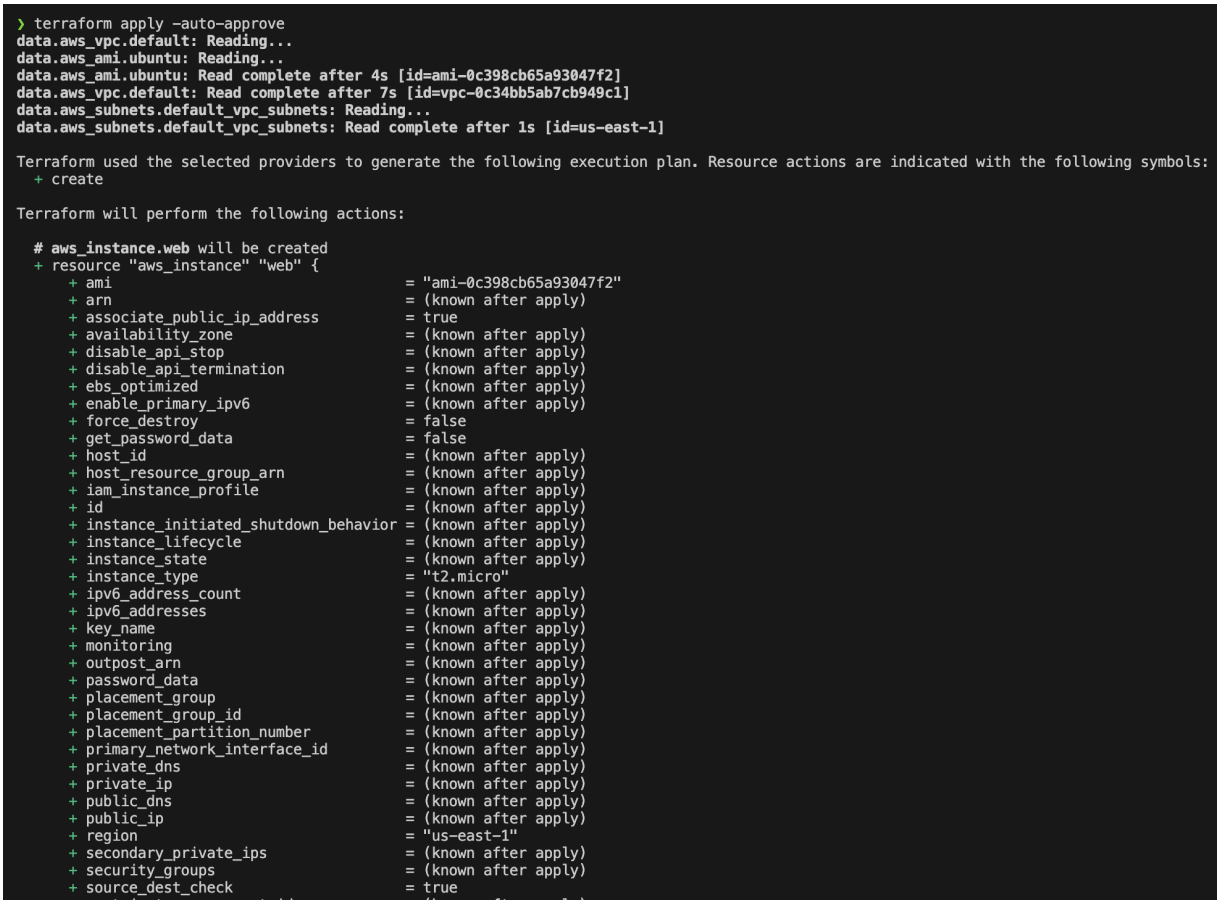


Abbildung 4: Terraform Apply

2 Workshop 2: Advanced Networking

In the second workshop, we move away from the default VPC and build a custom network infrastructure. This includes creating a VPC, subnets, routing tables, and an Internet Gateway. We also assign a persistent Elastic IP to our web server.

2.1 Network Infrastructure

2.1.1 VPC and Internet Gateway

We create a new VPC with the CIDR block `10.0.0.0/16`. To enable communication with the internet, we attach an Internet Gateway to this VPC.

```
resource "aws_vpc" "web_vpc" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}
```

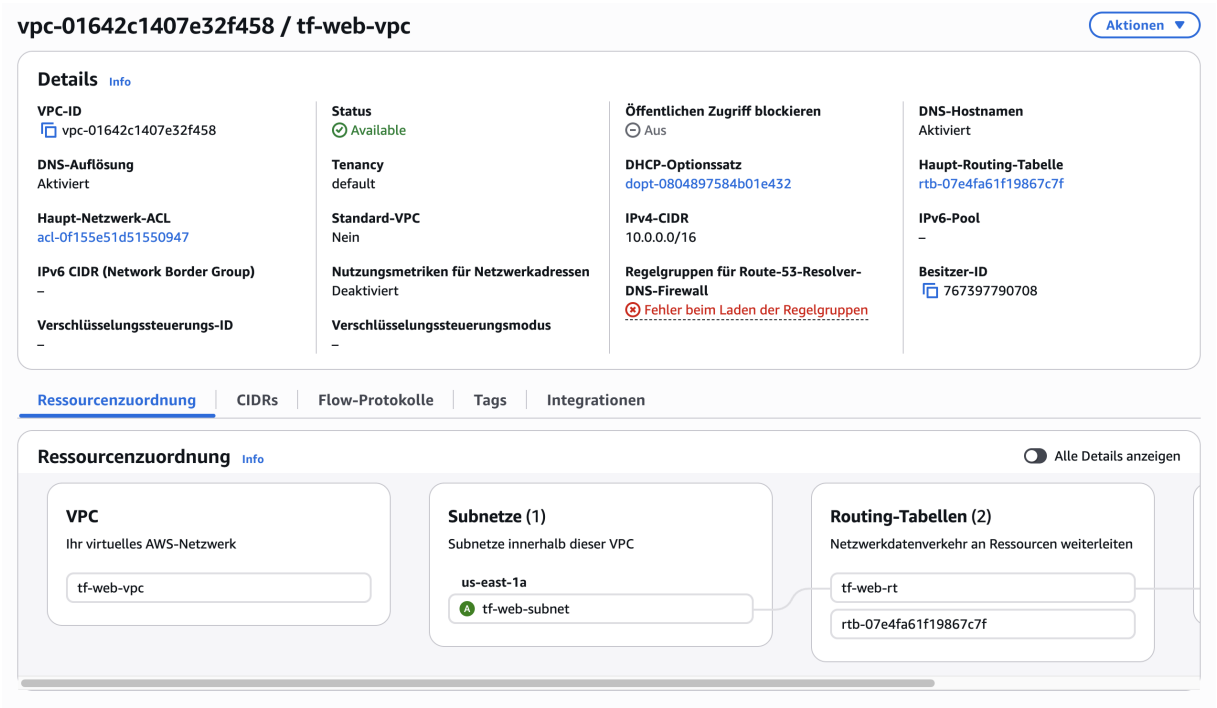


Abbildung 5: Custom VPC Created

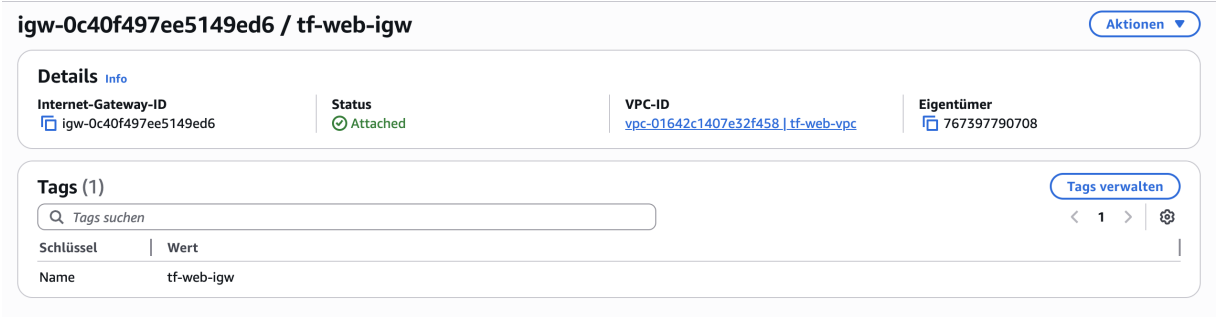


Abbildung 6: Internet Gateway Attached

2.1.2 Subnet and Routing

We create a subnet (`10.0.1.0/24`) within our VPC. To allow traffic from the subnet to reach the internet, we create a custom route table that routes all traffic (`0.0.0.0/0`) to the Internet Gateway, and associate it with our subnet.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_subnet" "web_subnet" {
  vpc_id                  = aws_vpc.web_vpc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = false
  tags = { Name = "tf-web-subnet" }
}

resource "aws_route_table_association" "web_rta" {
  subnet_id      = aws_subnet.web_subnet.id
  route_table_id = aws_route_table.web_rt.id
}
```

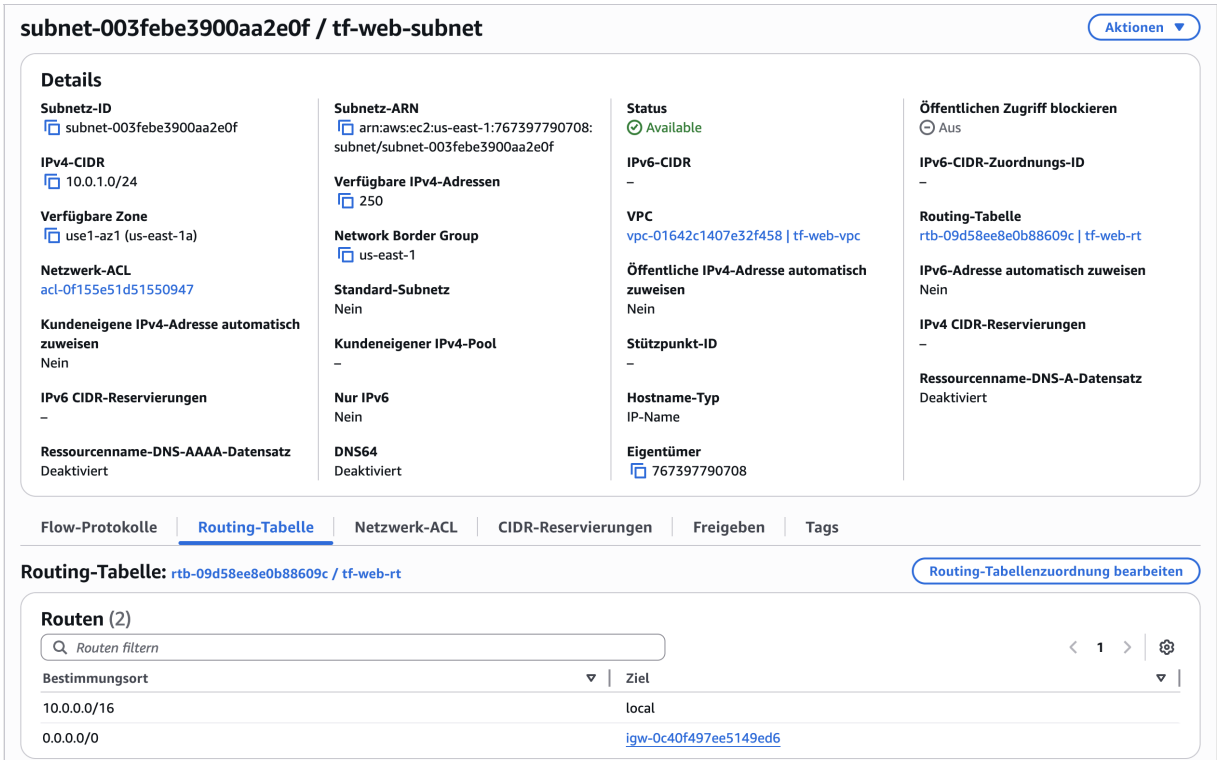


Abbildung 7: Subnet and Route Table Association

2.2 Compute and Access

2.2.1 Security Group

Similar to Workshop 1, we create a security group to allow HTTP traffic.

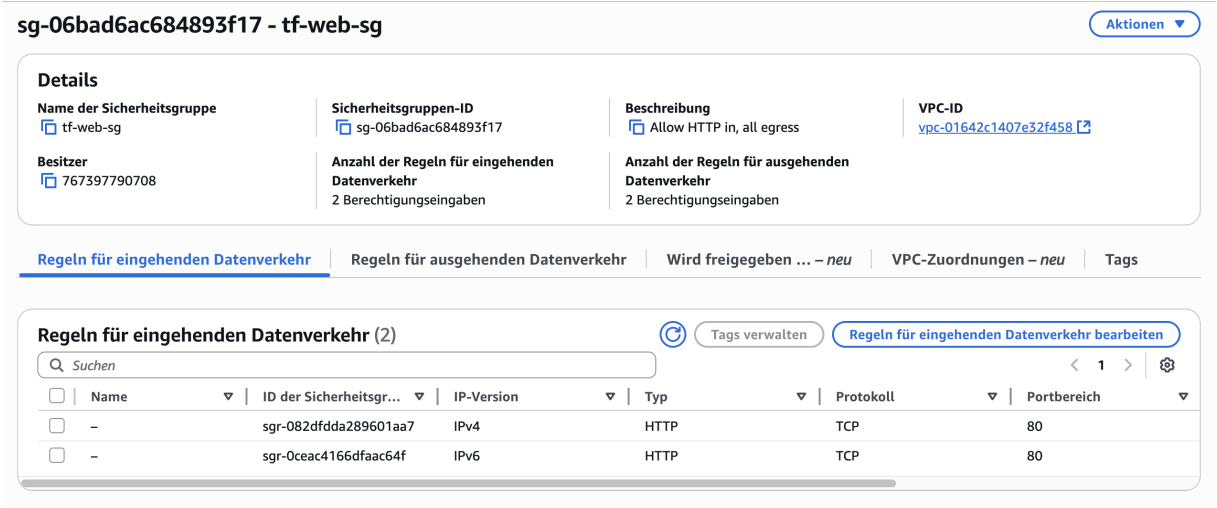


Abbildung 8: Security Group Inbound Rules

2.2.2 Network Interface and Elastic IP

Instead of relying on auto-assigned public IPs, we create a Network Interface (ENI) with a fixed private IP (`10.0.1.50`) and associate an Elastic IP (EIP) with it. This gives our server a static public IP address.

```
resource "aws_network_interface" "web_eni" {
  subnet_id       = aws_subnet.web_subnet.id
  private_ips     = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]
  tags = { Name = "tf-web-eni" }
}

resource "aws_eip" "web_eip" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.web_eni.id
  associate_with_private_ip = "10.0.1.50"
  depends_on                = [aws_internet_gateway.igw]
  tags = { Name = "tf-web-eip" }
}
```

Abbildung 9: Elastic IP Allocation

2.2.3 EC2 Instance

We launch the EC2 instance and attach the previously created network interface. This ensures the instance uses our specific network configuration.

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index         = 0
  }

  user_data = <<-EOF
    #!/bin/bash
    sudo apt-get update
```

```
    sudo apt-get install -y apache2
    sudo systemctl start apache2
    sudo systemctl enable apache2
    echo "<h1>Hello World</h1>" | sudo tee /var/www/html/index.html
  EOF

  tags = { Name = "tf-apache-instance" }
}
```

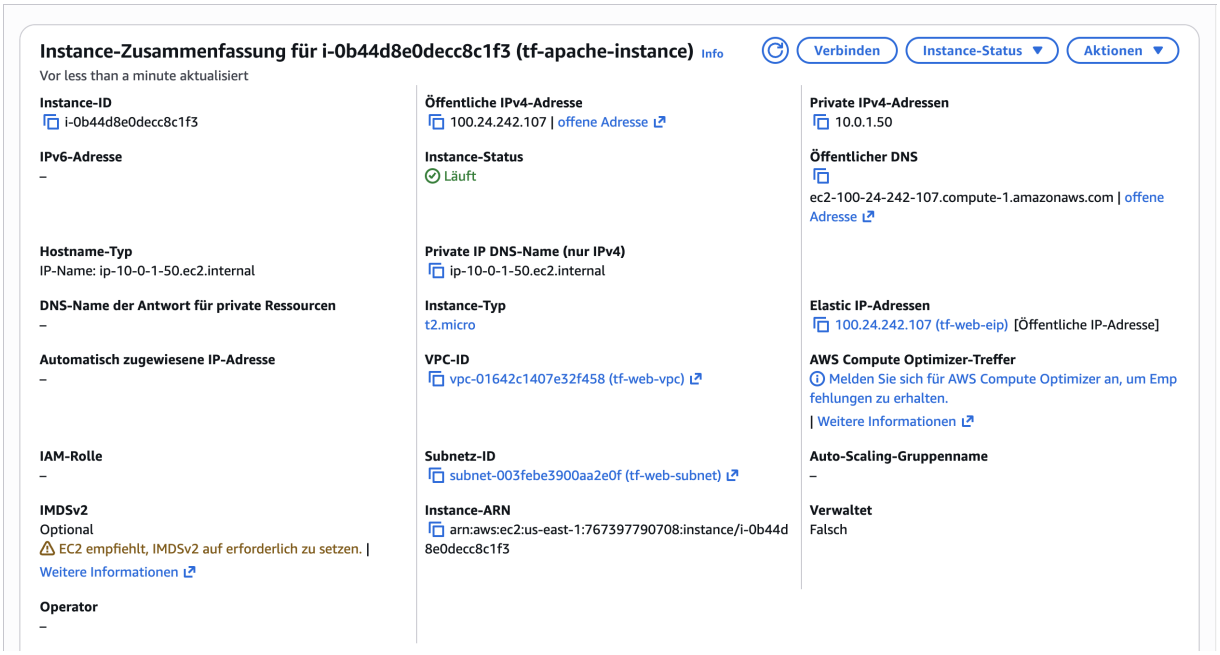


Abbildung 10: EC2 Instance Summary

2.3 Execution

We apply the configuration and verify the creation of resources. The output confirms the public IP of our instance.

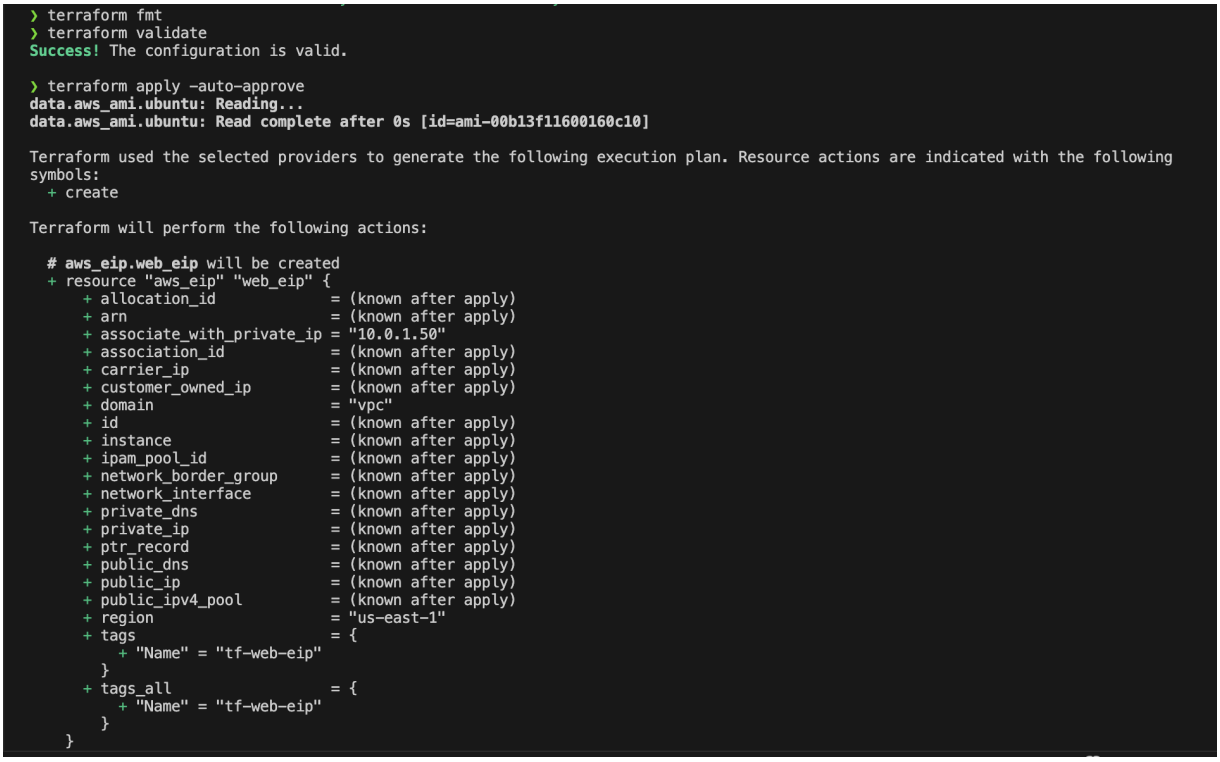


Abbildung 11: Terraform Apply Execution

3 Workshop 3: Load Balancing and High Availability

In this workshop, we implement a highly available web application architecture. We distribute traffic across two EC2 instances in different Availability Zones (AZs) using an Application Load Balancer (ALB). This ensures that if one AZ fails, the application remains accessible.

3.1 Networking Setup

3.1.1 VPC and Internet Gateway

We create a VPC and an Internet Gateway, similar to the previous workshop, to provide a network environment and internet connectivity.

```
resource "aws_vpc" "web_vpc" {
  cidr_block           = "10.0.0.0/16"
```

```
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}
```

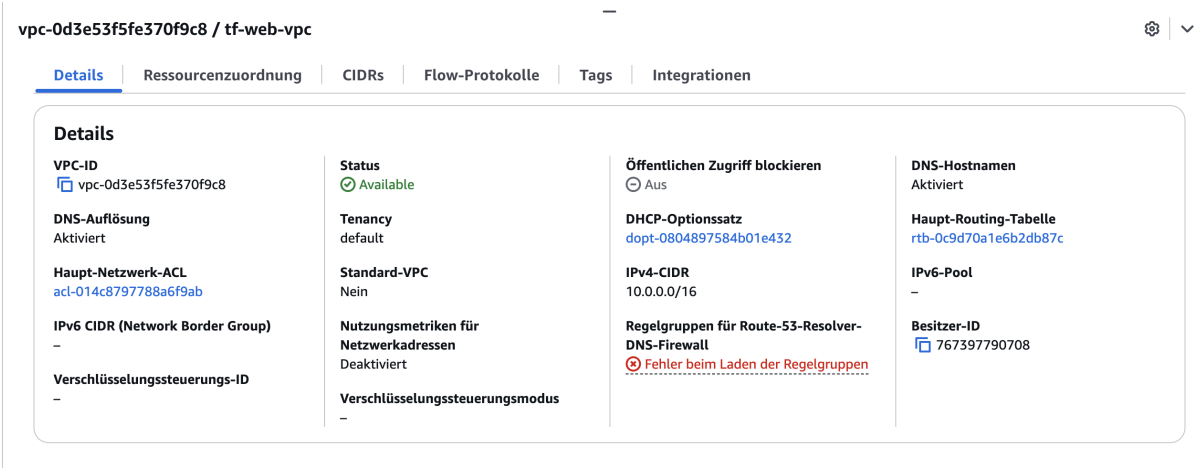


Abbildung 12: VPC Configuration

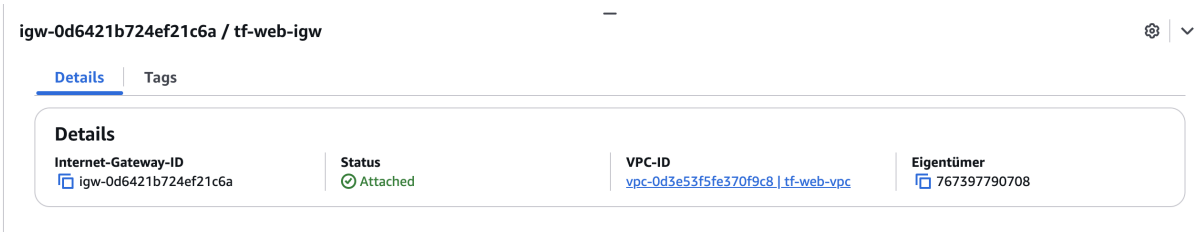


Abbildung 13: Internet Gateway

3.1.2 Subnets in Multiple AZs

To achieve high availability, we create two subnets in different Availability Zones (`us-east-1a` and `us-east-1b`).

```
resource "aws_subnet" "subnet_1" {
  vpc_id                  = aws_vpc.web_vpc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true
  tags = { Name = "tf-subnet-1" }
```

```
}

resource "aws_subnet" "subnet_2" {
  vpc_id                  = aws_vpc.web_vpc.id
  cidr_block              = "10.0.2.0/24"
  availability_zone       = "us-east-1b"
  map_public_ip_on_launch = true
  tags = { Name = "tf-subnet-2" }
}
```

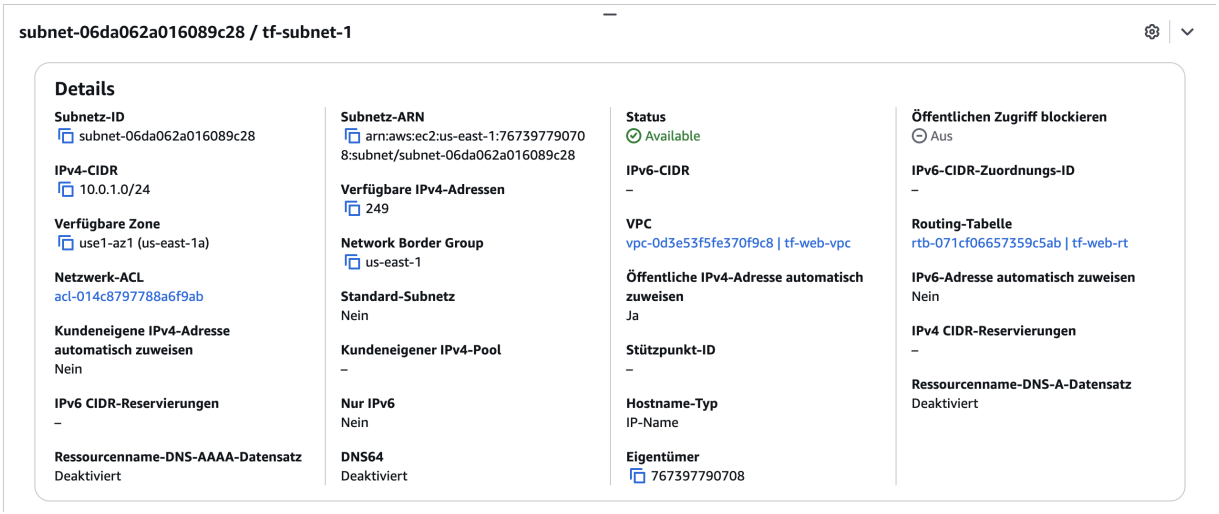


Abbildung 14: Subnet 1 in us-east-1a

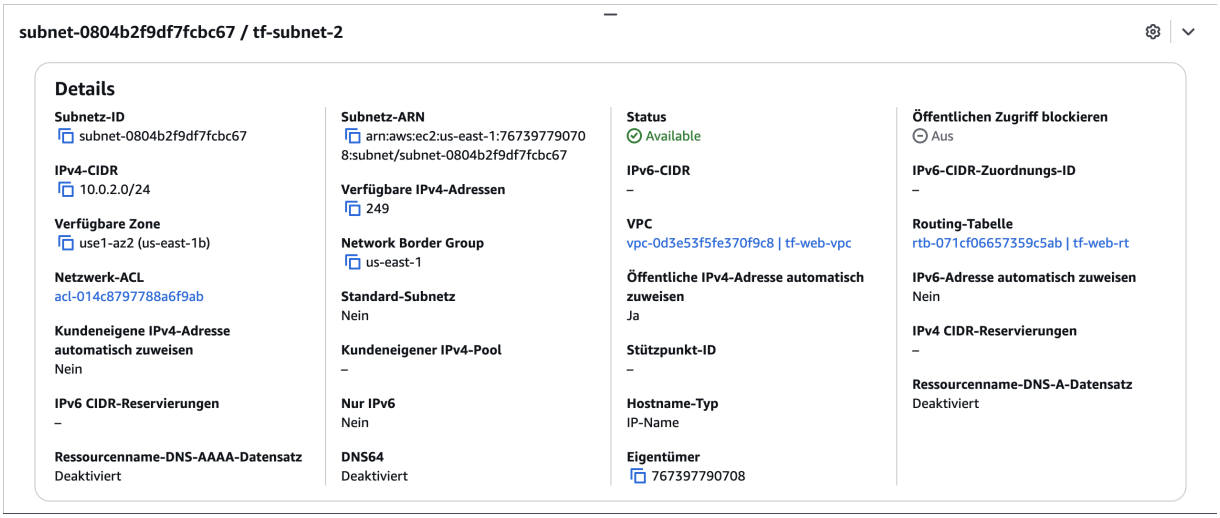


Abbildung 15: Subnet 2 in us-east-1b

3.1.3 Routing

We configure a route table to send internet traffic to the Internet Gateway and associate it with both subnets.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_route_table_association" "rta_1" {
  subnet_id      = aws_subnet.subnet_1.id
  route_table_id = aws_route_table.web_rt.id
}

resource "aws_route_table_association" "rta_2" {
  subnet_id      = aws_subnet.subnet_2.id
  route_table_id = aws_route_table.web_rt.id
}
```

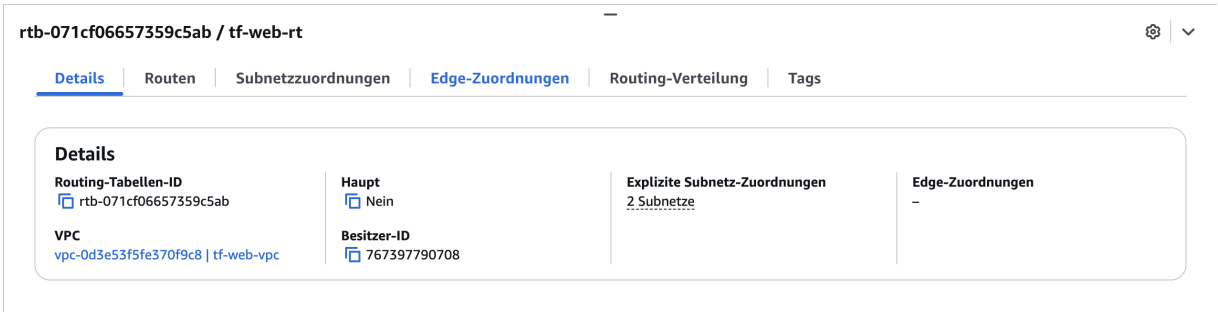


Abbildung 16: Route Table

Abbildung 17: Route Table Association

3.2 Security Groups

3.2.1 Load Balancer Security Group

This security group allows incoming HTTP traffic from anywhere to the Load Balancer.

```
resource "aws_security_group" "lb_sg" {
  name        = "tf-lb-sg"
  description = "Allow HTTP to Load Balancer"
  vpc_id      = aws_vpc.web_vpc.id

  ingress {
    description = "HTTP from Internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

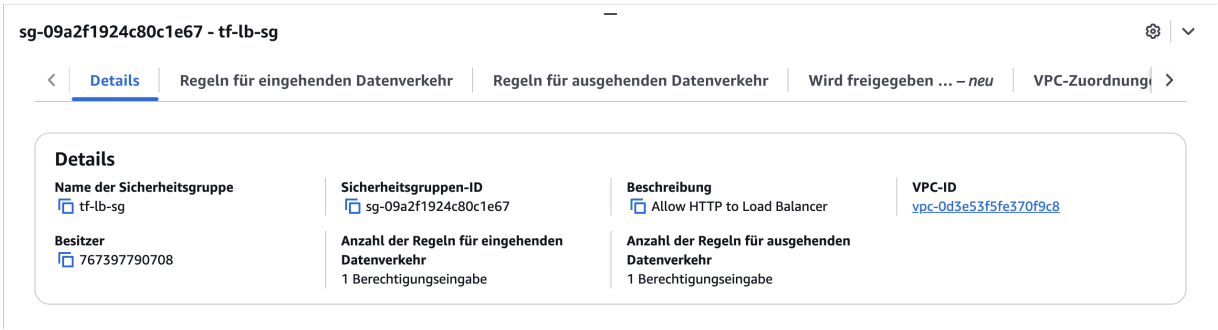


Abbildung 18: Load Balancer Security Group

3.2.2 EC2 Security Group

The EC2 instances should only accept traffic from the Load Balancer. We reference the Load Balancer's security group ID in the ingress rule.

```
resource "aws_security_group" "ec2_sg" {
  name        = "tf-ec2-sg"
```

```
  description = "Allow HTTP from LB"
  vpc_id      = aws_vpc.web_vpc.id

  ingress {
    description     = "HTTP from Load Balancer"
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.lb_sg.id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

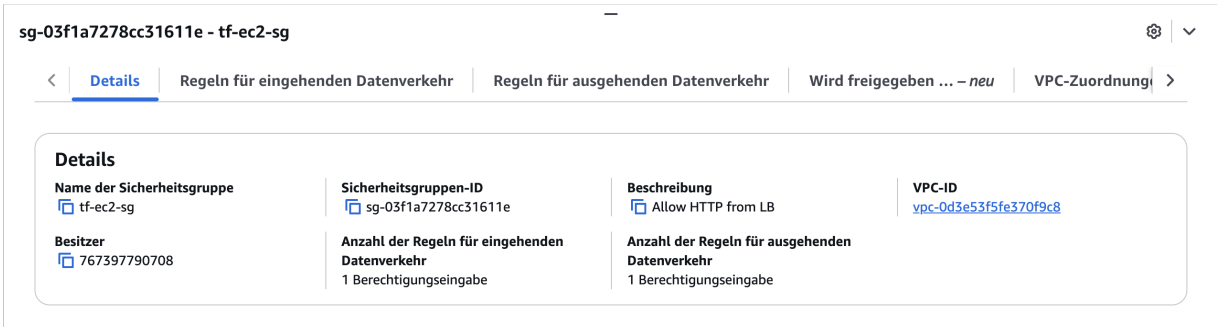


Abbildung 19: EC2 Security Group restricted to LB

3.3 Load Balancer Configuration

3.3.1 Application Load Balancer

We create an external Application Load Balancer spanning our two subnets.

```
resource "aws_lb" "app_lb" {
  name               = "tf-web-lb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.lb_sg.id]
  subnets            = [aws_subnet.subnet_1.id, aws_subnet.subnet_2.id]
  tags = { Name = "tf-web-lb" }
}
```

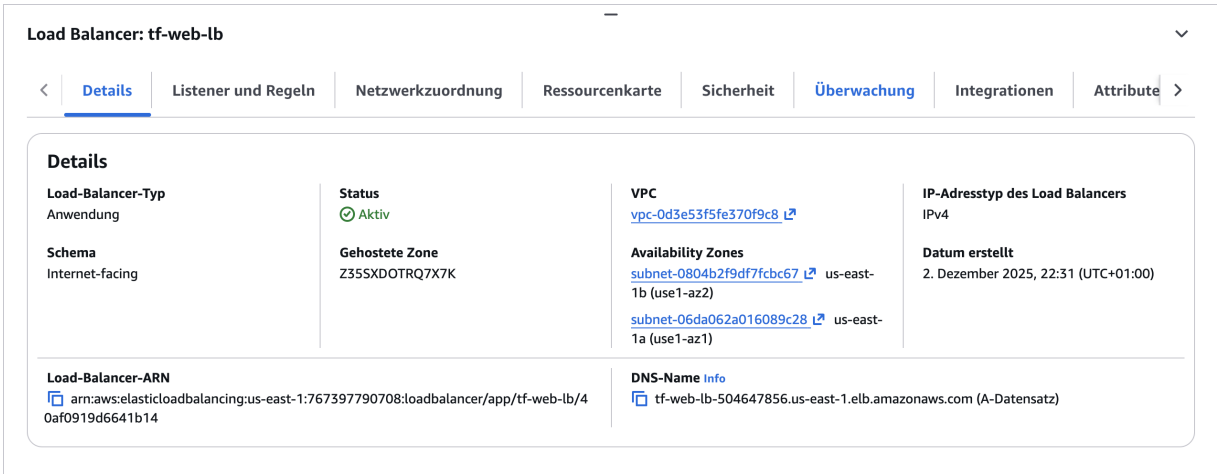


Abbildung 20: Application Load Balancer

3.3.2 Target Group and Listener

The target group defines where to route requests. The listener listens on port 80 and forwards traffic to this target group.

```
resource "aws_lb_target_group" "web_tg" {
  name     = "tf-web-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.web_vpc.id

  health_check {
    path                = "/"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 2
    unhealthy_threshold = 2
  }
}

resource "aws_lb_listener" "front_end" {
  load_balancer_arn = aws_lb.app_lb.arn
  port              = "80"
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.web_tg.arn
  }
}
```

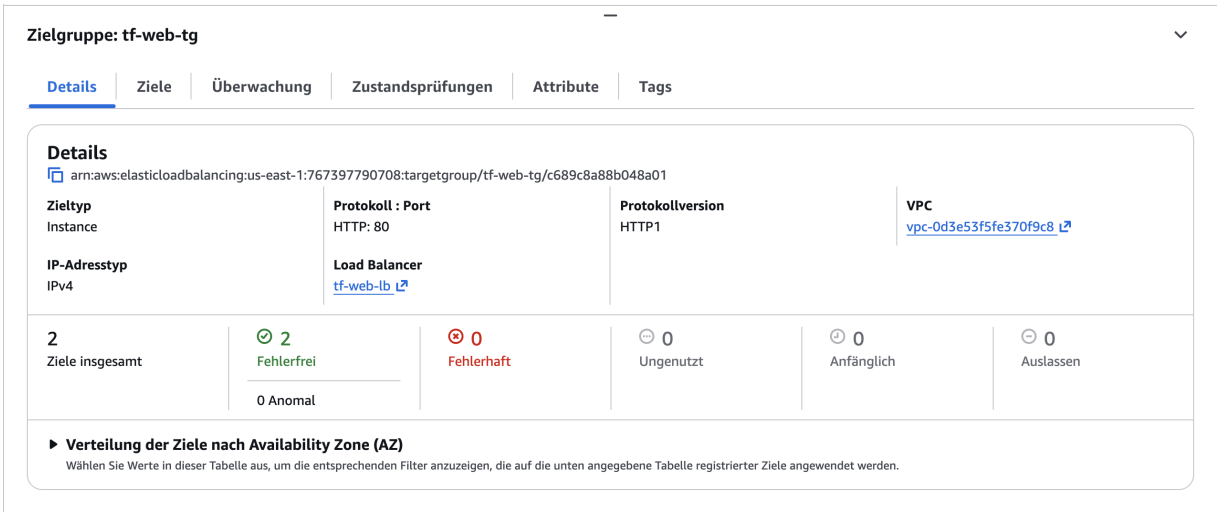


Abbildung 21: Target Group

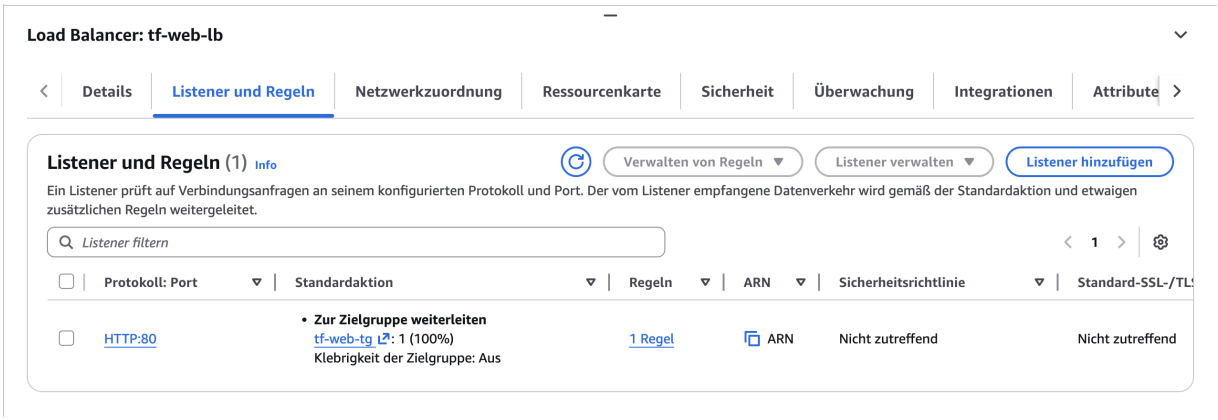


Abbildung 22: Listener Configuration

3.4 Compute Resources

3.4.1 EC2 Instances

We launch two EC2 instances. We use the `count` parameter to create multiple instances and the modulo operator to alternate them between the two subnets. The user data script is customized to display the instance number.

```
resource "aws_instance" "web" {
  count         = 2
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = count.index % 2 == 0 ? aws_subnet.subnet_1.id : aws_subnet
      .subnet_2.id
  vpc_security_group_ids = [aws_security_group.ec2_sg.id]
```

```
  user_data = <<-EOF
    #!/bin/bash
    sudo apt-get update
    sudo apt-get install -y apache2
    sudo systemctl start apache2
    sudo systemctl enable apache2
    echo "<h1>Hello␣from␣Instance␣${count.index␣+␣1}</h1>" | sudo tee /var/
        www/html/index.html
  EOF

  tags = { Name = "tf-web-instance-${count.index␣+␣1}" }
}
```

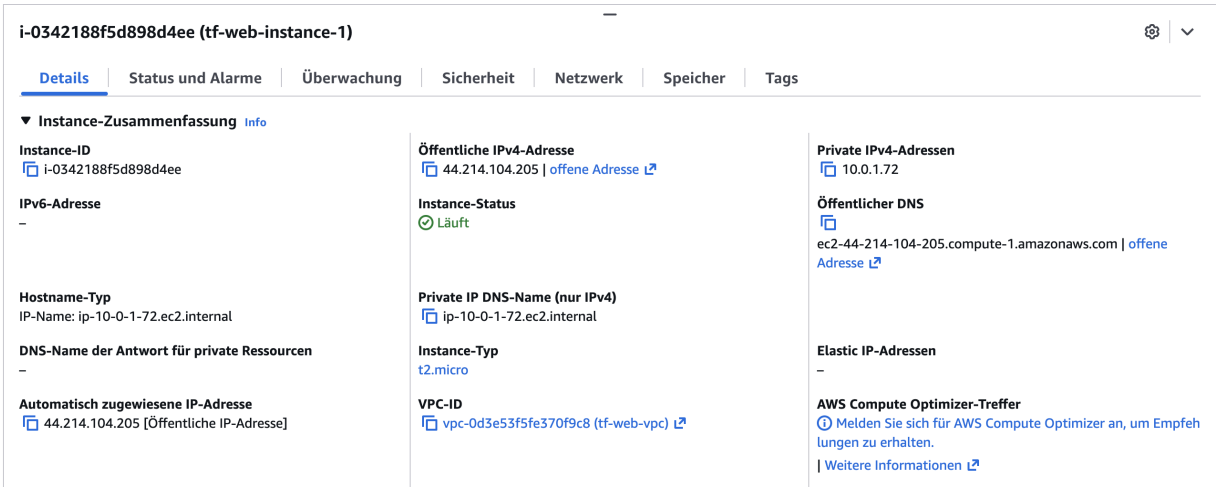


Abbildung 23: EC2 Instance 1

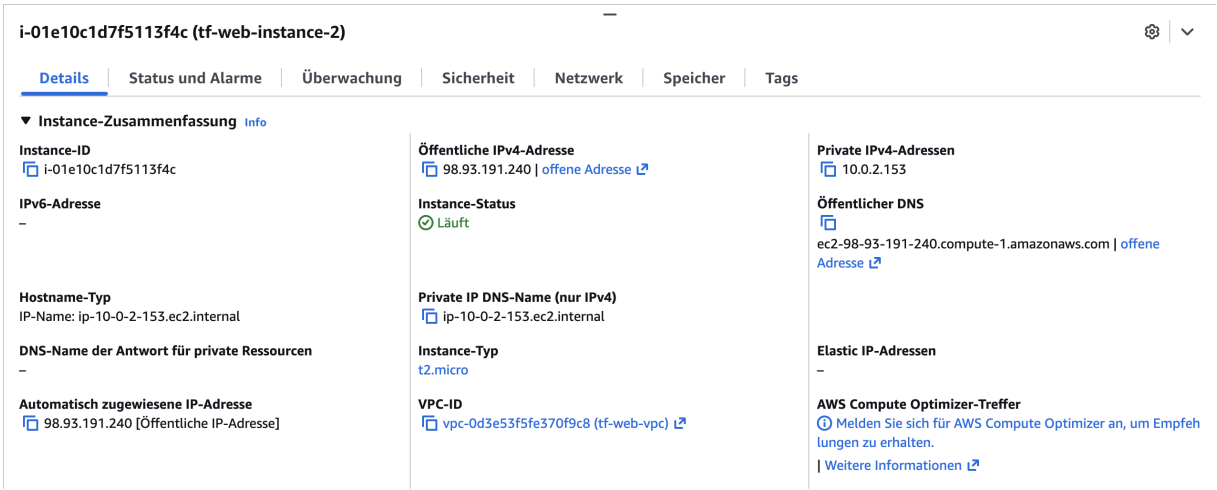


Abbildung 24: EC2 Instance 2

3.4.2 Target Group Attachment

We attach the created instances to the target group so the load balancer can route traffic to them.

```
resource "aws_lb_target_group_attachment" "web_attach" {
  count            = 2
  target_group_arn = aws_lb_target_group.web_tg.arn
  target_id        = aws_instance.web[count.index].id
  port             = 80
}
```

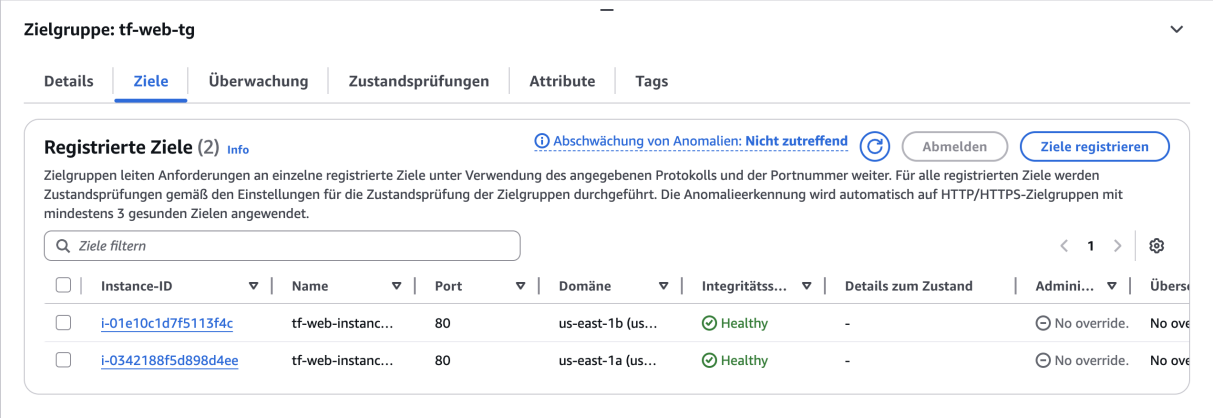


Abbildung 25: Target Group Attachment

3.5 Verification

After applying the configuration, we access the application via the Load Balancer's DNS name.

```
output "load_balancer_dns" {
  description = "The DNS name of the load balancer"
  value       = aws_lb.app_lb.dns_name
}
```

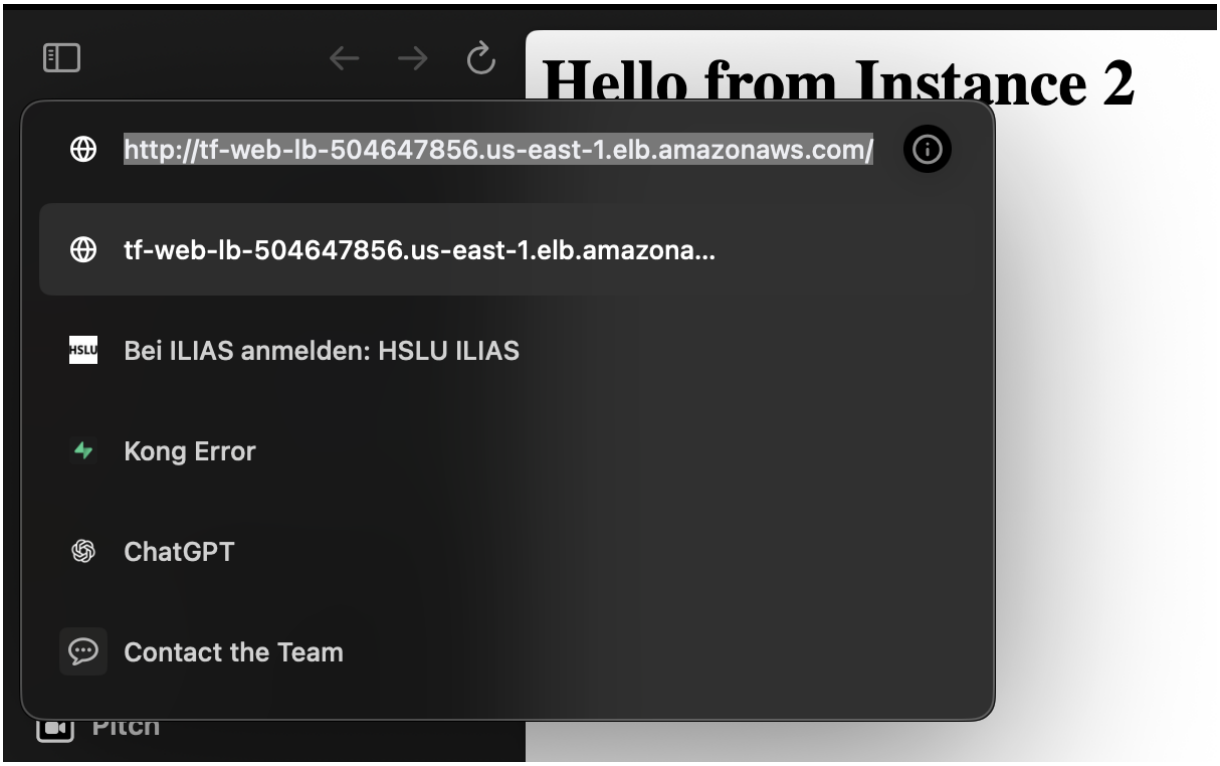


Abbildung 26: Website Accessed via Load Balancer

3.6 Automation and CI/CD

To automate the deployment process, we integrate GitHub Actions with Terraform Cloud. This allows us to automatically provision and manage our infrastructure upon code changes.

3.6.1 GitHub Configuration

We configure the necessary secrets in our GitHub repository to authenticate with Terraform Cloud.

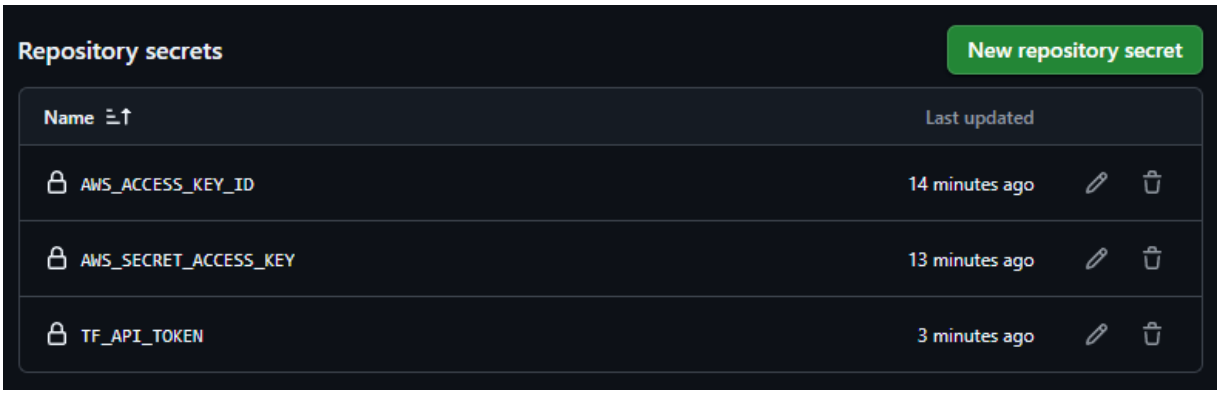


Abbildung 27: GitHub Repository Secrets

3.6.2 Terraform Cloud Configuration

In Terraform Cloud, we define the variables required for our infrastructure, such as AWS credentials and region settings.

Workspace variables (3)

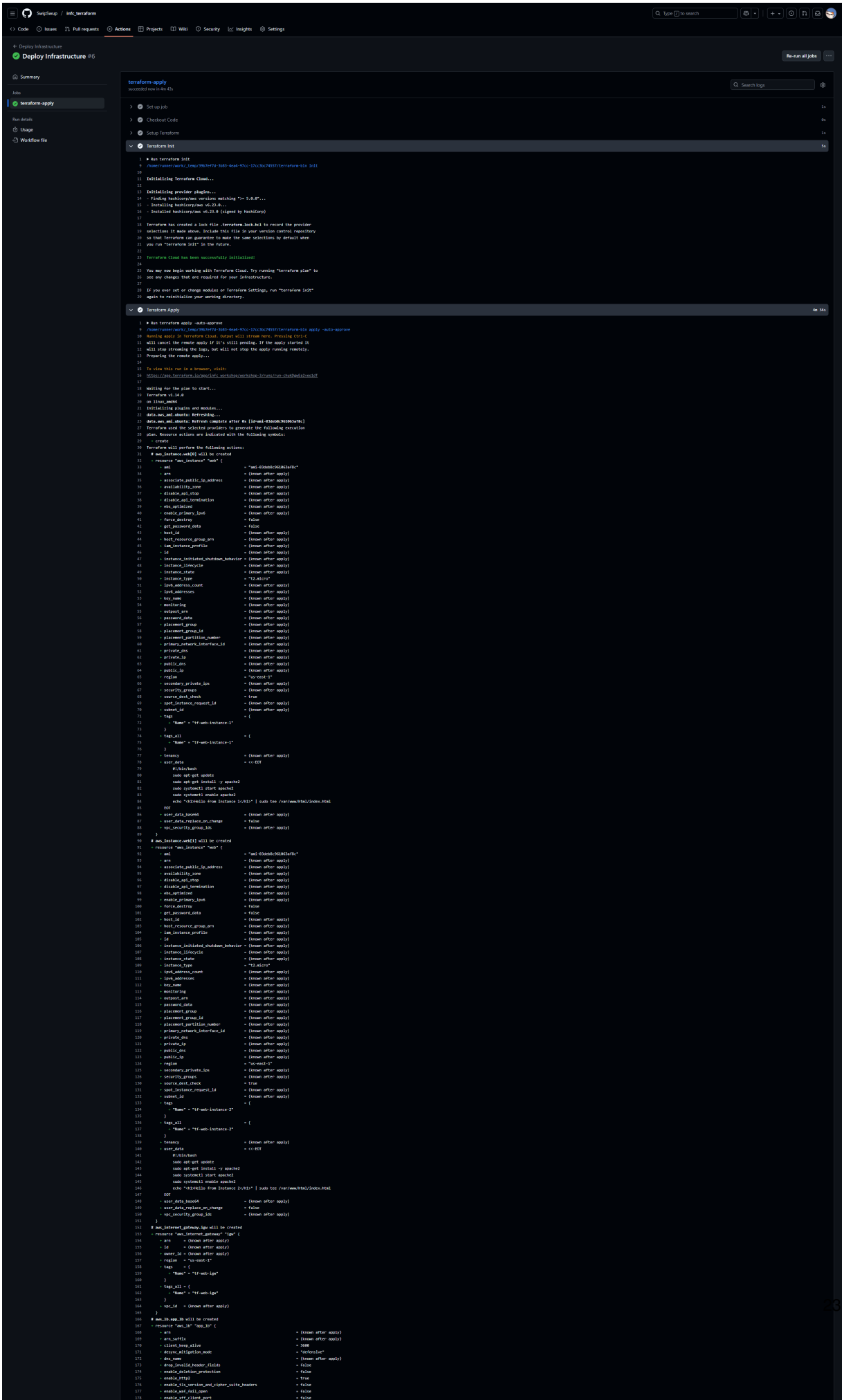
Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set precedence.

Key	Value	Category	Actions
AWS_ACCESS_KEY_ID	ASIA3FLDZAP2NU3F6LLY	env	...
AWS_SECRET_ACCESS_KEY	C02gmFSowFD7fesiEHM3usYdBKmJ92SqhtoeGosc	env	...
AWS_SESSION_TOKEN	IQoJb3JpZ2luX2VjEFEaCXVzLXdlc3QtMiJIMEYCIQD32A57GWmbeCvDmH71p1eCnIi25e4E8L+8H41eZfo89glhAMWbPrKRO+dFlc4JTOeTcWarw8RWAcNoIPA13b5lO74UKqYCCBoQABoMNzY3Mzk3NzkwNzA4Igysp0V0+91C2S6TAVYqgwIpB00+grqHu3U1IZLa6KCPb0mEGblYENUU40q/864udjJPFMkSnhfW1IVFQ0Neh4h3u2JtVkyqdQNdmKDFmze+PltxRg3S3r9cH6kUOw1YuKCOEzntBjFwdSUUIgkBc/oSKp/HQC7QzHox7XgZCHm8Jtn7CatNdNXdUesYZvE+XA6OUIAsZ3n5FoW6pSqcWunal66SBU6HP1njK9UljFYeDC3HsF183waztkRJdyhqERZriwu1vDJeZiKgW4o3iR/Nfo+5UNs86+ZT2wmd+0s3lkAfaQyxUb/zgkC0Rmtu1kkY3hRwRc1GEILFDk11bke/WfouZ9tmKBXNKkKbABU8M0K8MIe8vMkGOpwBwGXtYkoNeST2RixMNmiXpP7NXdH8j7KU/dVA6GdEGbXkT5OqvGo5+WBitfVq49MMoZCHLmhT8Y6FPt+wTtp/pm3QILTU1kKEXhdvXy2Rdlyb/X5ZcsRekPQtBN2mp4oEW5RTCKwLyRCK4ztMfLYw4Ej/dvUZ+qrx9Lbn9UybIJKtYfebpMGWhhIkqUW1ez2c2SRtheUqtJ0oXXGn	env	...

Abbildung 28: Terraform Cloud Variables

3.6.3 CI/CD Pipeline

When code is pushed to the repository, a GitHub Action is triggered. This action communicates with Terraform Cloud to plan and apply the changes.


SwipSwup / infc_terraform
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Deploy Infrastructure
Deploy Infrastructure #6
Re-run all jobs
Summary
Jobs
terraform-apply
Run details
Usage
Workflow file
terraform-apply
succeeded now in 4m 43s
Search logs
Set up job
Checkout Code
Setup Terraform
Terraform Init
Run terraform init
/home/runner/work/_temp/39b7ef7d-3683-4ea4-97cc-17cc3bc24557/terraform-bin init
Initializing Terraform Cloud...
Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0.0"...
- Installing hashicorp/aws v6.23.0...
- Installed hashicorp/aws v6.23.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
Terraform Cloud has been successfully initialized!
You may now begin working with Terraform Cloud. Try running "terraform plan" to
see any changes that are required for your infrastructure.
If you ever set or change modules or Terraform Settings, run "terraform init"
again to reinitialize your working directory.
Terraform Apply
4m 34s
Run terraform apply -auto-approve
/home/runner/work/_temp/39b7ef7d-3683-4ea4-97cc-17cc3bc24557/terraform-bin apply -auto-approve
Running apply in Terraform Cloud. Output will stream here. Pressing Ctrl-C
will cancel the remote apply if it's still pending. If the apply started it
will stop streaming the logs, but will not stop the apply running remotely.
Preparing the remote apply...
To view this run in a browser, visit:
https://app.terraform.io/app/infc_workshop/workshop-3/runs/run-ch6KQwEa2vnoldf
Waiting for the plan to start...
Terraform v1.14.0
on linux_amd64
Initializing plugins and modules...
data.aws_ami.ubuntu: Refreshing...
data.aws_ami.ubuntu: Refresh complete after 0s [id=ami-03deb8c961063ef8c]
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
# aws_instance.web[0] will be created
+ resource "aws_instance" "web" {
+ ami = "ami-03deb8c961063ef8c"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
+ force_destroy = false
+ get_password_data = false
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
+ id = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle = (known after apply)
+ instance_state = (known after apply)
+ instance_type = "t2.micro"
+ ipv6_address_count = (known after apply)
+ ipv6_addresses = (known after apply)
+ key_name = (known after apply)
+ monitoring = (known after apply)
+ outpost_arn = (known after apply)
+ password_data = (known after apply)
+ placement_group = (known after apply)
+ placement_group_id = (known after apply)
+ placement_partition_number = (known after apply)
+ primary_network_interface_id = (known after apply)
+ private_dns = (known after apply)
+ private_ip = (known after apply)
+ public_dns = (known after apply)
+ public_ip = (known after apply)
+ region = "us-east-1"
+ secondary_private_ips = (known after apply)
+ security_groups = (known after apply)
+ source_dest_check = true
+ spot_instance_request_id = (known after apply)
+ subnet_id = (known after apply)
+ tags = {
"Name" = "tf-web-instance-1"
}
+ tags_all = {
"Name" = "tf-web-instance-1"
}
+ tenancy = (known after apply)
+ user_data = <<-EOT
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello from Instance 1</h1>" | sudo tee /var/www/html/index.html
EOT
+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids = (known after apply)
}
# aws_instance.web[1] will be created
+ resource "aws_instance" "web" {
+ ami = "ami-03deb8c961063ef8c"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
+ force_destroy = false
+ get_password_data = false
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
+ id = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle = (known after apply)
+ instance_state = (known after apply)
+ instance_type = "t2.micro"
+ ipv6_address_count = (known after apply)
+ ipv6_addresses = (known after apply)
+ key_name = (known after apply)
+ monitoring = (known after apply)
+ outpost_arn = (known after apply)
+ password_data = (known after apply)
+ placement_group = (known after apply)
+ placement_group_id = (known after apply)
+ placement_partition_number = (known after apply)
+ primary_network_interface_id = (known after apply)
+ private_dns = (known after apply)
+ private_ip = (known after apply)
+ public_dns = (known after apply)
+ public_ip = (known after apply)
+ region = "us-east-1"
+ secondary_private_ips = (known after apply)
+ security_groups = (known after apply)
+ source_dest_check = true
+ spot_instance_request_id = (known after apply)
+ subnet_id = (known after apply)
+ tags = {
"Name" = "tf-web-instance-2"
}
+ tags_all = {
"Name" = "tf-web-instance-2"
}
+ tenancy = (known after apply)
+ user_data = <<-EOT
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello from Instance 2</h1>" | sudo tee /var/www/html/index.html
EOT
+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids = (known after apply)
}
# aws_internet_gateway.igw will be created
+ resource "aws_internet_gateway" "igw" {
+ arn = (known after apply)
+ id = (known after apply)
+ owner_id = (known after apply)
+ region = "us-east-1"
+ tags = {
"Name" = "tf-web-igw"
}
+ tags_all = {
"Name" = "tf-web-igw"
}
+ vpc_id = (known after apply)
}
# aws_lb.app_lb will be created
+ resource "aws_lb" "app_lb" {
+ arn = (known after apply)
+ arn_suffix = (known after apply)
+ client_keep_alive = 3600
+ desync_mitigation_mode = "defensive"
+ dns_name = (known after apply)
+ drop_invalid_header_fields = false
+ enable_deletion_protection = false
+ enable_http2 = true
+ enable_tls_version_and_cipher_suite_headers = false
+ enable_waf_fail_open = false
+ enable_xff_client_port = false

The execution of the plan can be monitored in Terraform Cloud.

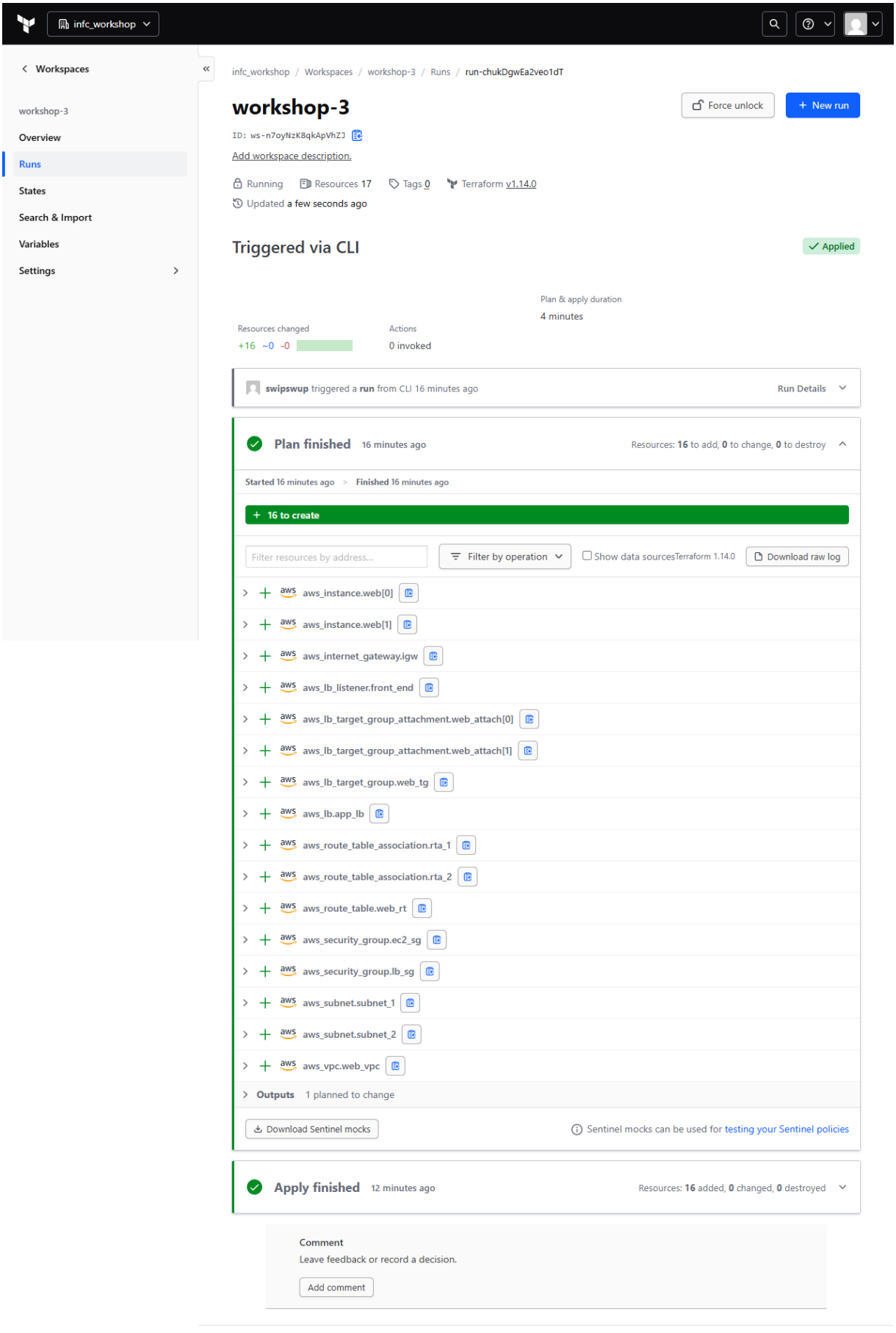


Abbildung 30: Terraform Cloud Apply Run

Finally, we can also manage the destruction of resources through the pipeline or Terraform Cloud directly.

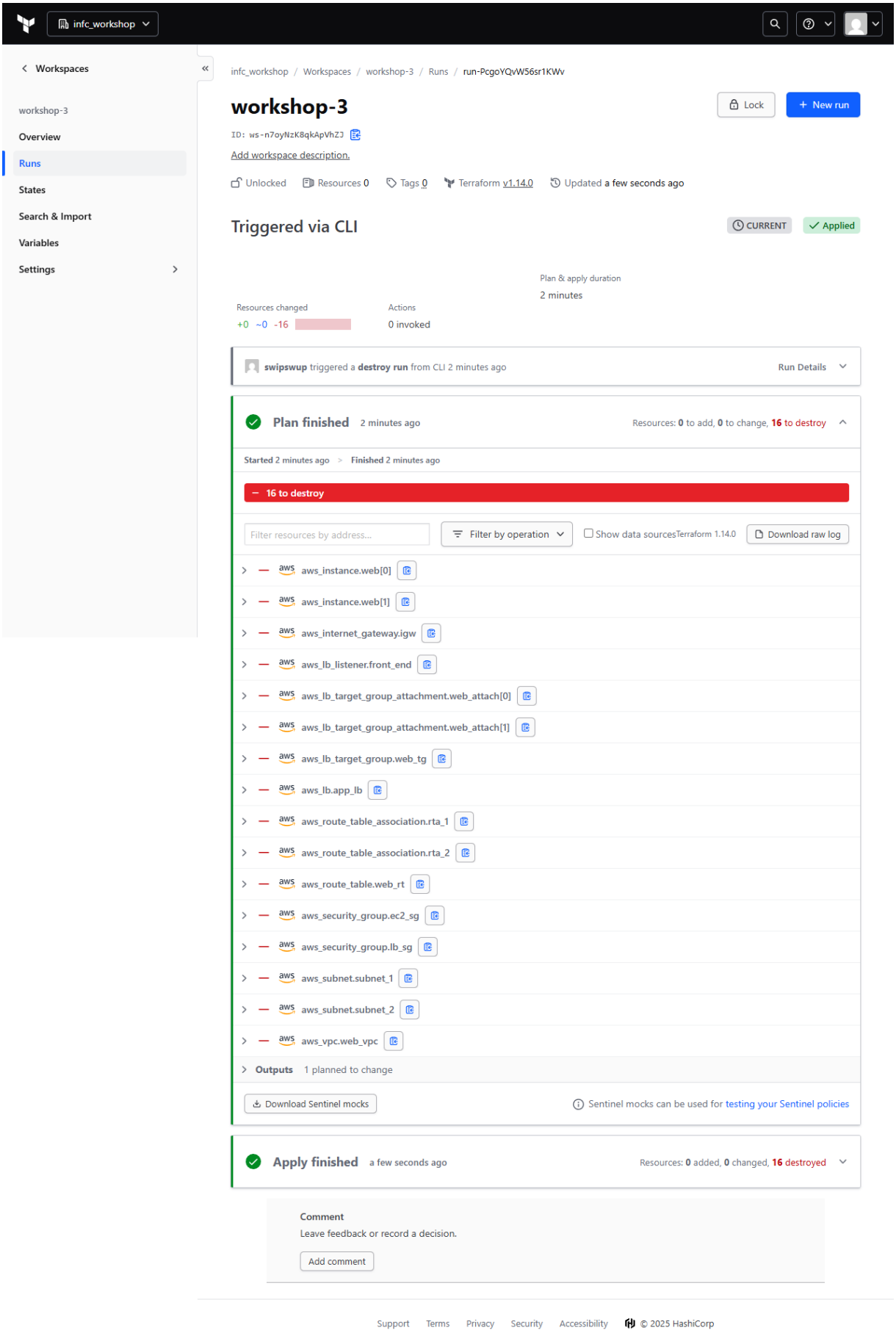


Abbildung 31: Terraform Cloud Destroy Run